HDI Prediction Using Machine Learning

Project Description:

The Human Development Index (HDI) Predictor is a Machine Learning-based web application that predicts the Human Development Index of a country using key socio-economic indicators. The application analyzes four important input parameters: Life Expectancy at Birth, Expected Years of Schooling, Mean Years of Schooling, and Gross National Income (GNI) per Capita.

The system is built using Python, Flask, Scikit-learn, Pandas, and HTML/CSS. A trained Machine Learning regression model predicts the HDI value based on user input. The application provides quick and accurate predictions through an easy-to-use web interface, helping users understand the relationship between development indicators and the Human Development Index.

Scenarios

Scenario 1:

A government analyst enters the country's Life Expectancy, Expected Years of Schooling, Mean Years of Schooling, and GNI per Capita. The system predicts the country's Human Development Index and displays the estimated HDI score instantly.

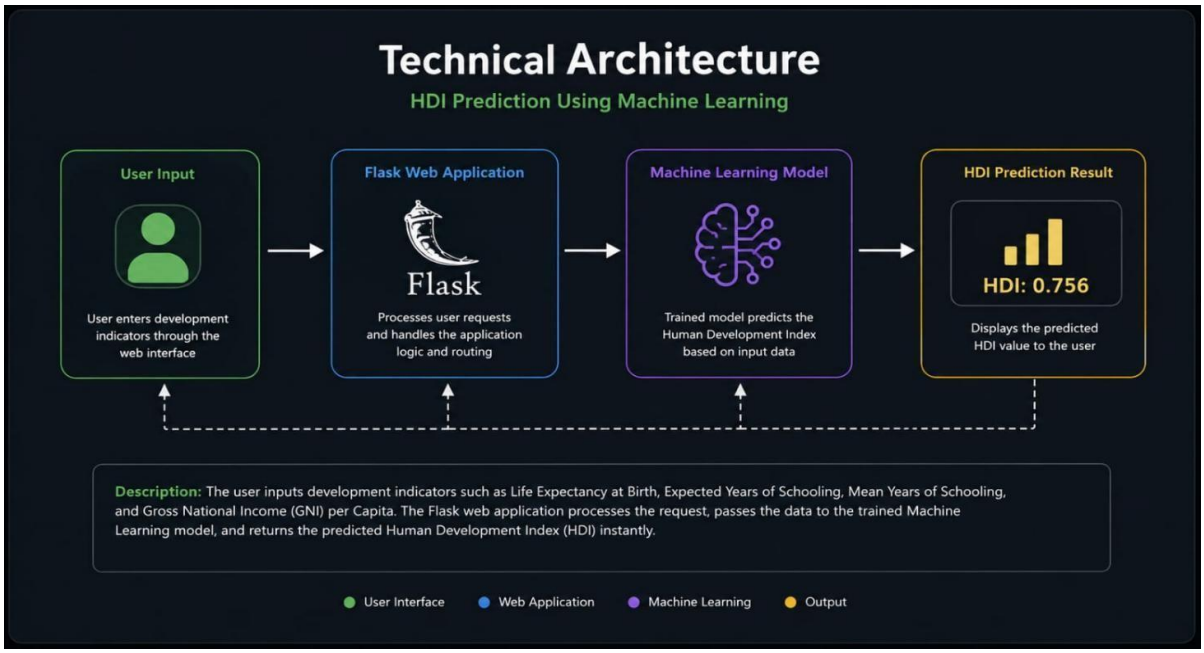
Scenario 2:

A researcher compares development indicators of multiple countries by entering different values. The application predicts HDI for each dataset, making comparative analysis easier.

Scenario 3:

A student learning Machine Learning experiments with different socio-economic values to observe how changes in education, income, and life expectancy affect the Human Development Index prediction.

Technical Architecture:



Description:

The HDI Predictor follows a three-layer architecture.

-Frontend: HTML, CSS

-Backend: Flask (Python)

-Machine Learning Layer: Scikit-learn Regression Model trained using HDI dataset

The user enters the development indicators through the web interface. Flask validates the input and sends it to the trained Machine Learning model. The model predicts the Human Development Index and returns the result, which is displayed on the webpage.

Pre-requisites:

- VS Code
- **Python Programming**
- Flask Framework
- Machine Learning Fundamental
- Pandas
- NumPy
- Scikit-Learn
- HTML, CSS

Technology Stack:

- Frontend: HTML, CSS, Bootstrap
- Backend: Python, Flask
- Machine Learning: Scikit-learn
- Libraries: Pandas, NumPy
- IDE: Visual Studio Code

Project Workflow:

Milestone 1: Dataset Collection & Model Selection

- **Activity 1.1:** Collect the Human Development Index dataset.
- **Activity 1.2:** perform data preprocessing and cleaning.
- **Activity 1.3:** Select suitable Machine Learning Regression algorithm.
- **Activity 1.4:** Split dataset into Training and Testing data.

Milestone 2: Machine Learning Model Development

- **Activity 2.1:** Train the regression model using the HDI dataset.
- **Activity 2.2:** Evaluate model performance using appropriate evaluation metrics.
- **Activity 2.**3: Save the trained model for deployment.

Milestone 3: Flask Application Development

- **Activity 3.1: Create the Flask application (app.py).**
- **Activity 3.2: Load the trained Machine Learning model.**
- **Activity 3.3: Receive user input and generate HDI prediction**

Milestone 4: Frontend Development

- **Activity 4.1: Design responsive web pages using HTML and CSS.**

- **Activity 4.2: Create input forms for development indicators.**
- **Activity 4.3: Display the predicted HDI value on the webpage.**

Milestone 5: Deployment

- **Activity 5.1: Test the complete application locally.**
- **Activity 5.2: Deploy the Flask application using Render or another cloud platform.**
- **Activity 5.3: Verify prediction accuracy and application performance.**

Milestone 6: Conclusion

The HDI Prediction system successfully predicts the Human Development Index using Machine Learning techniques. The project demonstrates how AI can assist in analyzing development indicators and generating accurate HDI predictions through a user-friendly web application.

Milestone 1: Model Selection and Architecture

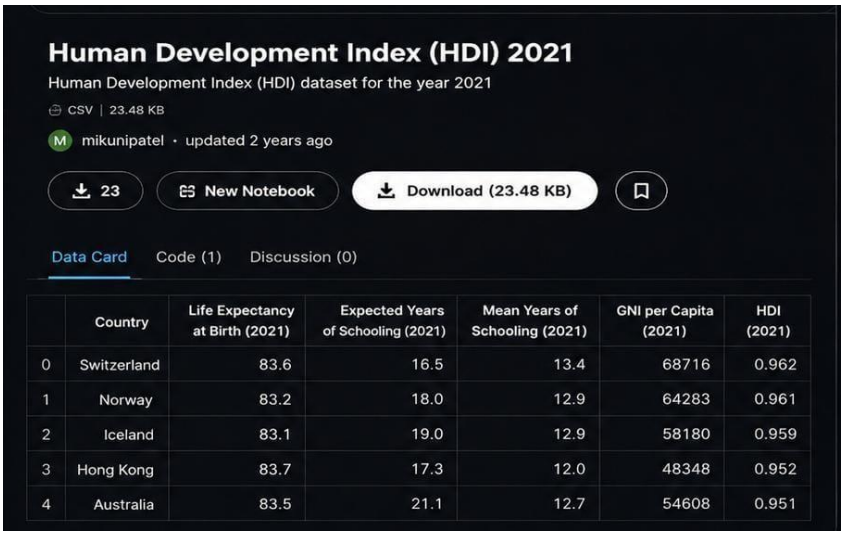
In this milestone, we focus on selecting the most suitable Machine Learning algorithm for predicting the Human Development Index (HDI). We analyze the dataset, preprocess the data, choose the best regression model, and design the overall application architecture for accurate HDI prediction.

Activity 1.1: Collect and Prepare the Dataset

Before training the model, obtain a reliable Human Development Index dataset.

- **Step 1 — Download the Dataset**

  Download the HDI dataset from Kaggle or the UNDP Human Development Reports.



Human Development Index (HDI) 2021

Human Development Index (HDI) dataset for the year 2021

CSV | 23.48 KB

mikunipatel · updated 2 years ago

23 | New Notebook | Download (23.48 KB)

Data Card | Code (1) | Discussion (0)

	Country	Life Expectancy at Birth (2021)	Expected Years of Schooling (2021)	Mean Years of Schooling (2021)	GNI per Capita (2021)	HDI (2021)
0	Switzerland	83.6	16.5	13.4	68716	0.962
1	Norway	83.2	18.0	12.9	64283	0.961
2	Iceland	83.1	19.0	12.9	58180	0.959
3	Hong Kong	83.7	17.3	12.0	48348	0.952
4	Australia	83.5	21.1	12.7	54608	0.951

- **Step 2 — Load the Dataset**

  Load the dataset using the Pandas library.

- **Step 3 — Explore the Dataset**

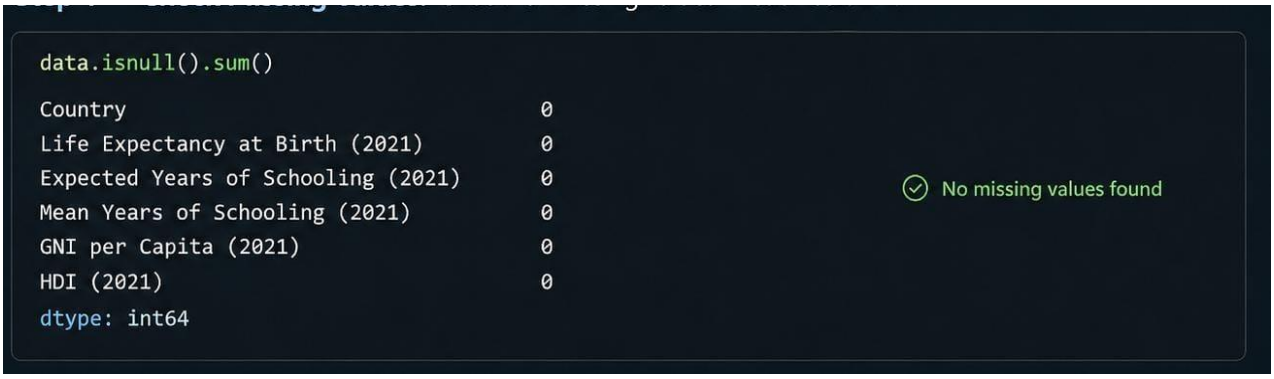
Display the first few rows and understand the available features.

```
data.head()
```

	Country	Life Expectancy at Birth (2021)	Expected Years of Schooling (2021)	Mean Years of Schooling (2021)	GNI per Capita (2021)	HDI (2021)
0	Switzerland	83.6	16.5	13.4	68716	0.962
1	Norway	83.2	18.0	12.9	64283	0.961
2	Iceland	83.1	19.0	12.9	58180	0.959
3	Hong Kong	83.7	17.3	12.0	48348	0.952
4	Australia	83.5	21.1	12.7	54608	0.951

- **Step 4 — Handle Missing Values**

  Remove or fill missing values to improve model accuracy.



- **Step 5 — Select Features**

Input Features:

- Life Expectancy at Birth (2021)
- Expected years of Schooling (2021)
- Mean Years of Schooling (2021)
- Gross National Income per Capita (2021)

Target:

- Human Development Index (2021)

Activity 1.2: Research and Select the Appropriate Machine Learning Model

Consider the following before selecting the model:

- Understand the HDI prediction problem.
- Analyze relationships between socio-economic indicators.
- Compare different regression algorithms.
- Evaluate model using R6^2 score and Mean Squared Error.
- Select the best-performing model (e.g., Random Forest Regressor or Linear Regression).

Activity 1.3: Define the Application Architecture

- Design the system architecture.
- **Develop the HTML/CSS fronted.**
- **Build the Flask backend.**
- **Integrate the trained Machine Learning model.**
- **Display predicted HDI on the result page.**

Activity 1.4: Set Up the Development Environment

- Install the required software and libraries.
- Python 3.x

- Visual Studio Code
- Flask
- Pandas
- NumPy
- Scikit-learn
- Joblib
- Install libraries using:

```
pip install pandas
pip install numpy
pip install scikit-learn
pip install flask
pip install matplotlib
pip install joblib
```

- Create Virtual Environment:

```
python -m veny myenv
myenv\Scripts\activate
```

- Create Virtual Environment:

```
Pip install -r requirements.txt
```

Project Structure

```
HDI_Predictor/
│
├── static/
```

```
│   ├── css/
│   └── images/
│
├── templates/
│   ├── index.html
│   └── result.html
│
├── model.pkl
├── app.py
├── train_model.py
├── requirements.txt
└── dataset.csv
```

Milestone 2: Core Functionalities Development

Activity 2.1: Develop Core Machine Learning Features

Implement the core functionalities required for Human Development Index prediction using Machine Learning.

• Data preprocessing:

Clean the dataset by handling missing values, removing duplicates, and selecting relevant features for training.

• Features Selection: Select the four major input features:

- Life Expectancy at Birth (2021)
- Expected years of Schooling (2021)
- Mean Years of Schooling (2021)
- Gross National Income (GNI) per Capita (2021)

•Model Training: Train the Machine Learning regression model using the prepared datasetvand evaluate its prediction accuracy.

• HDI Prediction: Use the trained model to predict the Human Development Index (HDI) based on user-provided inputs.

Activity 2.2: Implement the Flask Backend

Define Routes in Flask:

- Create route in app.py for the Home page and HDI Predictor page.
- Handle GET and POST requests using Flask.

Process User Input

- Create an HTML form for entering the four development indicators.
- Validate all user inputs before prediction.
- Convert input values into the required numerical format.

Integrated Machine Learning Model

- Load the trained model. pkl file using Joblib.
- Pass user input to the trained model.
- Generate the predicted HDI value.
- Display the prediction result on the webpage.

Milestone 3: App.py Development

Milestone 3 focuses on creating and refining the core logic of the Human Development Index (HDI) Predictor application. In this milestone, the Flask application loads the trained Machine Learning model, accepts user inputs, predicts the Human Development Index, and displays the result through a web interface.

Activity 3.1: Create the Main Application Logic in app.py

Define the Core Routes in app.py

Establish two main routes for the HDI Predictor application.

/- Home Page Route

Display the index.html page where users enter the required development indicators.

```
@app.route('/')
def home():
   return render_template('index.html')
```

/predict – Prediction Route

Receive user input, loads the trained Machine Learning model, predicts the HDI value, and displays the result

```
@app.route('/predict', methods=['POST'])
def predict():

   life_expectancy = float(request.form['life_expectancy'])
   expected_schooling =
float(request.form['expected_schooling'])
   mean_schooling = float(request.form['mean_schooling'])
   gni = float(request.form['gni'])

   prediction = model.predict([[life_expectancy,
                        expected_schooling,
                        mean_schooling,
                        gni]])

   return render_template("result.html",
                     prediction=prediction[0])
```

Set Up Route Handlers for Each Feature:

- Read the four input values from index.html.
- Validate that all fields contain valid numeric values.
- Pass the inputs to the trained Machine Learning model.
- Display the predicted Human Development Index on the result page.

Load the Machine Learning Model

Load the saved Machine Learning model using Joblib.

```
Import joblib
Model = joblib.load("model.pkl")
```

Input Validation

Validate user input before prediction.

```
If life_expectancy <=0:
    Return "Invalid Life Expectancy"

If gni <=0:
    Return "Invalid Income"
```

Generate HDI Prediction

Predict the Human Development Index.

```
Prediction = model.predict([[life_expectancy, expected_schooling, mean_schooling, gni]])
```

Display Prediction

Return the predicted HDI value to the result page.

```
return render_template(
    "result.html",
     Prediction=prediction[0]
```

```
)
```

Application flow

- User opens the Home Page.
- User enters:
  - Life Expectancy at Birth
  - Expected Years of Schooling
  - Mean Years of Schooling
  - Gross National Income per Capita
- Flask validates the input.
- The trained Machine Learning model predicts the HDI.
- The predicted Human Development Index is displayed on the webpage.

Milestone 4: Frontend Development

Milestone 4 focuses on developing a user-friendly and responsive web interface for the Human Development Index (HDI) Predictor. This milestone includes designing HTML pages, styling them with CSS, and displaying the predicted HDI result dynamically using Flask templates.

Activity 4.1: Designing and Developing the User Interface

Set Up the Base HTML Structure:

- Develop the main index.html page as the primary interface.
- Create input fields for:
- Life Expectancy at Birth
- Expected Years of Schooling
- Mean Years of Schooling
- Gross National Income (GNI) per Capita
- Add a Predict HDI button to submit the form.
- Display the predicted HDI value in a separate result section.

Design a Responsive Layout Using CSS:

- Create a responsive webpage using HTML, CSS, and Bootstrap.
- Use a clean and professional layout for desktop and mobile devices.
- Add proper spacing, labels, colors, and buttons for better user experience.
- Ensure the interface adapts to different screen sizes using responsive design.

Create Separate UI Components for Each Output Type:

- Input Form for entering HDI parameters.
- Prediction Result Card to display the predicted HDI value.
- Error Message Section for invalid inputs.
- Navigation Header with the project title.

Activity 4.2: Creating Dynamic Content Rendering with JavaScript

Handle Form Submission Asynchronously:

- Validate all input fields before submission.
- Prevent empty or invalid values.
- Send user input to the Flask backend using a POST request.
- Display a loading message while the prediction is being processed.

Render Results Dynamically:

- Receive the predicted HDI value from the Flask backend.
- Display the prediction without reloading the page (optional if using JavaScript).
- Show a success message after prediction.

Restore UI State After Generation:

- Re-enable the Predict HDI button after prediction.
- Clear the loading indicator.
- Display an error message if prediction fails.
- Allow users to perform multiple predictions without refreshing the page.

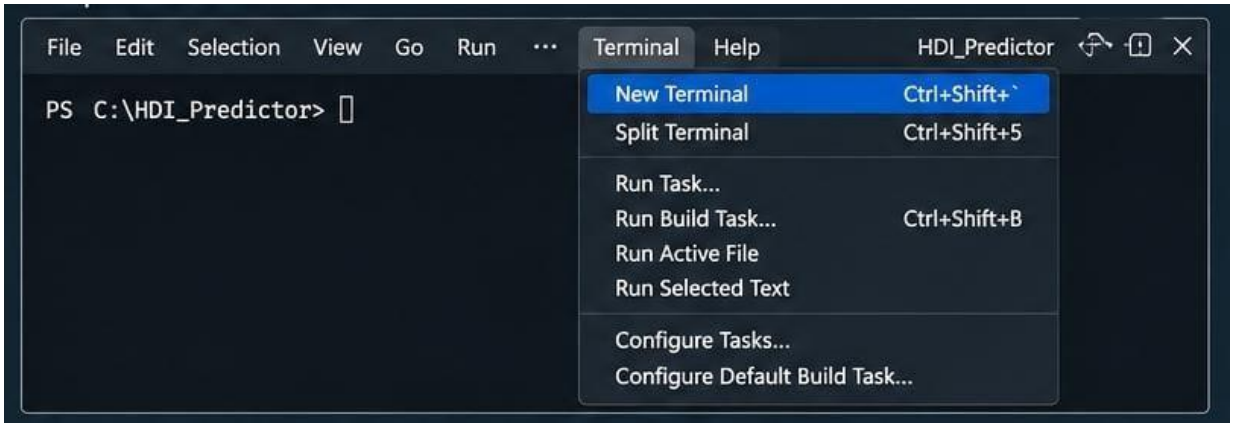
Milestone 5: Deployment

In Milestone 5, the focus is on deploying the Human Development Index (HDI) Predictor application first locally to verify correct operation, and then publicly via ngrok to share the app over a secure, accessible URL. This involves configuring the server environment, managing dependencies, and launching the app fir real-world use.

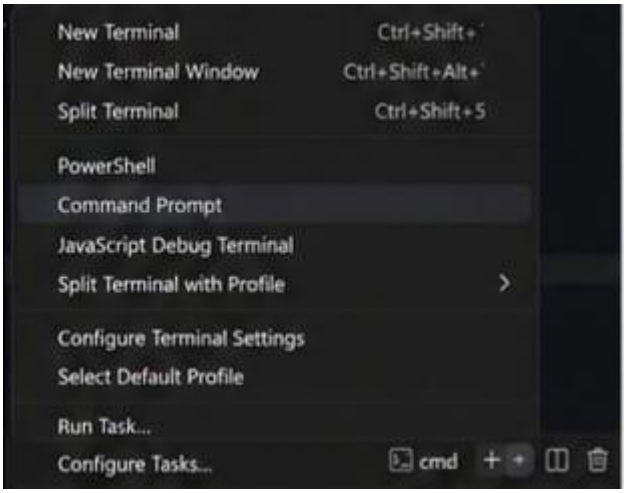
Activity 5.1: Preparing the Application for Local Deployment

Set Up a Virtual Environment:

- Open a New Terminal



- Then open "Command Prompt"



Create and activate a virtual environment, then install all required dependcies from requirement. Txt.

```
D:\HDI_Predictor>python -m venv myenv

D:\HDI_Predictor>myenv\Scripts\activate

(myenv) D:\HDI_Predictor>pip install -r requirements.txt
```

Configure Environment Variables:

- Create a .env file in the project root and add any necessary configuration (if required). The app reads this automatically using python-dotenv.

```
FLASK_ENV=development
SECRET_KEY=your_secret_key_here
```

Activity 5.2: Local Testing and Verification

Start the Flask Development Server:

- Run the application locally using:

```
(venv) D:\HDI_Predictoe> python app.py
```

- Once running, open a browser and navigate to "http://127.0.0.1:5000" to access the app.

```
(venv) D:\HDI_Predictor> python app.py
 * Serving Flask app 'app'
 * Debug mode: off
 WARNING: This is a development server. Do not use it in a production deployment.
 * Running on http://127.0.0.1:5000
Press CTRL+C to quit
 * Restarting with stat
 * Debugger is active!
 * Debugger PIN: 638-821-426
```

- Enter different values for the four indicators and verify that the predicted HDI value is displayed correctly.

- Ensure the model and UI are working as expected.

Activity 5.3: Public Deployment via Ngrok

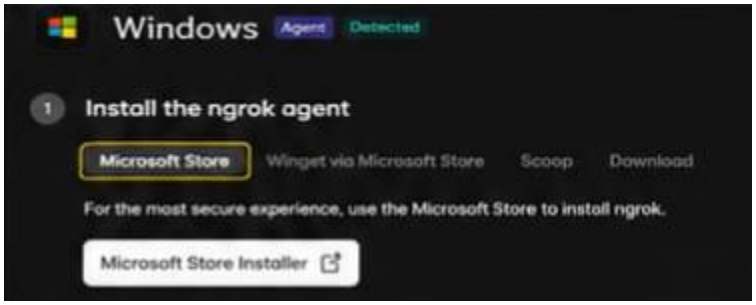
Ngrok creates a secure tunnel from a public URL to your local Flask server, making the HDI Predictor application accessible from anywhere without a cloud hosting provider.

Install Ngrok and pyngrok:

- Install the pyngrok Python wrapper .

```
(myenv) D:\HDI_Predictor>pip install pyngrok
```

- Download and install the Ngrok client from https://ngrok.com/download.
- Click on "Microsoft Store Installer".



Configure Your Ngrok Authtoken:

- sign up at ngrok.com and copy your authtoken from the dashboard.
- The run_public.py script sets the authtoken automatically using:

```
(myenv) D:\HDI_Predictor>ngrok config add-authtoken YOUR_NGROK_AUTHTOKEN
```

- Replace the YOUR_NGROK_AUTHTOKEN value in run_public.py with your own ngrok authtoken before running.

Run the Public Deployment Script:

- Run run_public.py to start both the Ngrok tunnel and Flask server.

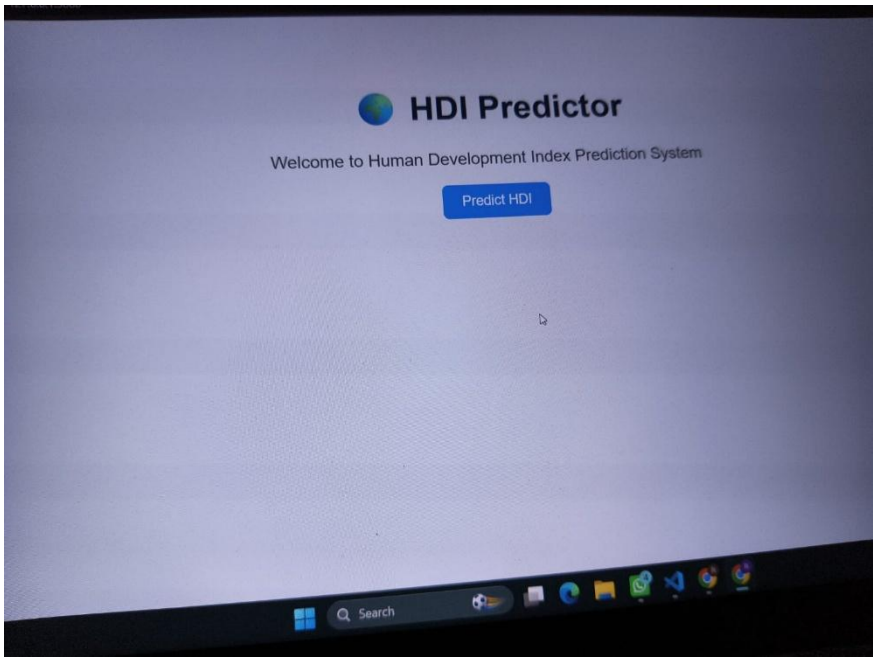
```
(myenv) D:\HDI_Predictor>python run_public.py
```

- The script opend an HTTP tunnel on port 5000 and prints the live public URL in the terminal.

```
============================================================
YOUR HDI PREDICTOR APP IS LIVE!
URL: https://abc123.ngrok-free.app
============================================================
```

Exploring the Website's Web Pages:

Home / Generator Page:

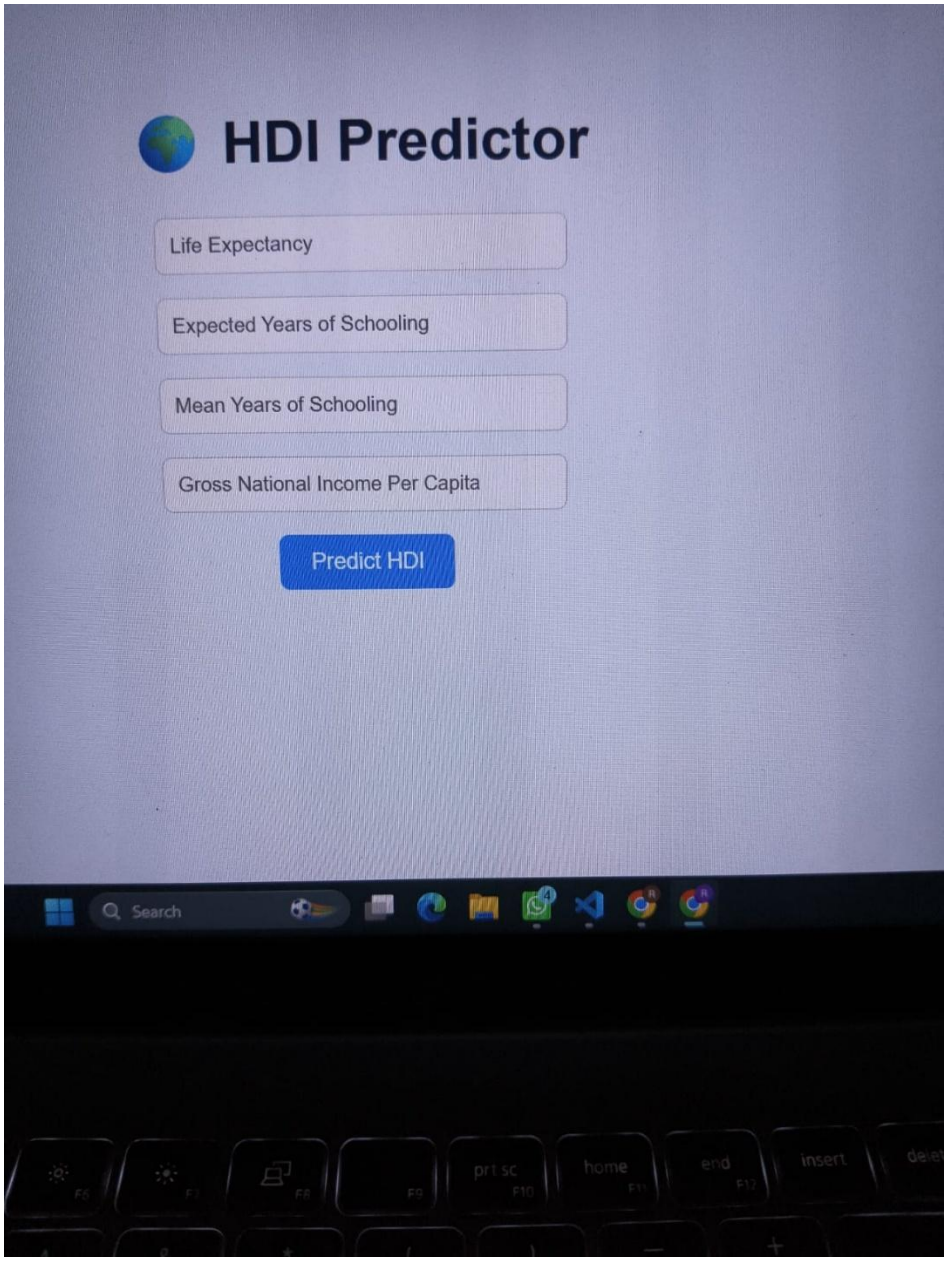


Description:

The Home Page welcome users to the Human Development Index (HDI) Prediction System.Users can click the Predict HDI button to navigate to the prediction pag

2. Prediction Page / Input Section:

Input Section



Description:

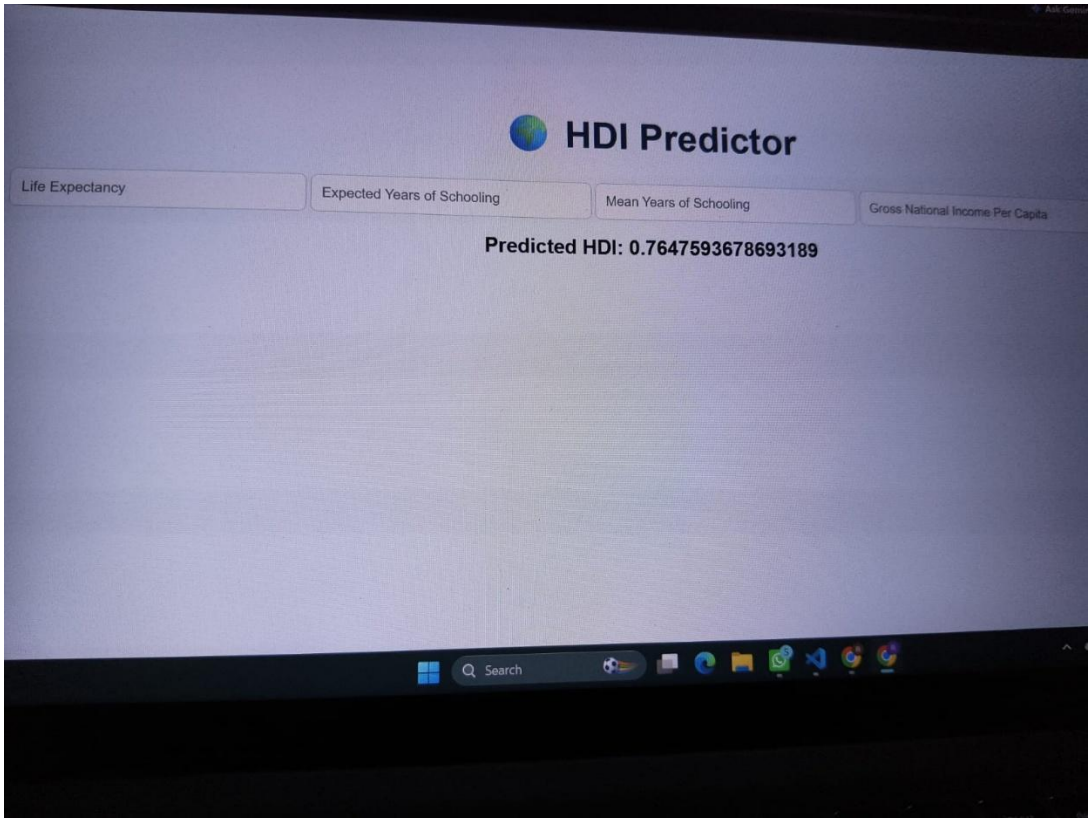
The input Section allows users to enter the four development indicators:

- Life Expectancy
- Expected Years of Schooling
- Mean Years of Schooling
- Gross National Income Per Capita

After entering the values, users click Predict HDI to generate the prediction.

3. Results Section

Results Section



Description:

The Results Section displays the predicted Human Development Index (HDI) generated by the trained Machine Learning model based on the user's input values.

Conclusion:

The Human Development Index (HDI) Predictor Using Machine Learning is a web-based application developed to predict the Human Development Index based on important socio-economic indicators such as Life Expectancy at Birth, Expected Years of Schooling, Mean Years of Schooling, and Gross National Income (GNI) per Capita. The project uses a trained Machine Learning regression model to provide accurate and reliable HDI predictions through a simple and user-friendly web interface.

The application was developed using Python, Flask, Scikit-learn, HTML, and CSS, demonstrating how Machine Learning can be applied to solve real-world development and socio-economic prediction problems. It provides fast predictions, easy data input, and an interactive interface, making it useful for students, researchers, and policymakers.

In the future, the project can be enhanced by adding country-wise comparisons, graphical data visualization, real-time dataset updates, cloud deployment, and support for additional development indicators, making the application more informative, scalable, and practical for wider use.